

What We Ran, and What We Found

A reduced version of the Zivich & Breskin cross-fit study

Six estimators, machine-learning nuisance models, 1,000 simulated datasets
Paper reproduced: *Machine learning for causal inference: on the use of cross-fit estimators*,
Epidemiology 2021 (arXiv:2004.10337)

July 27, 2026

The one-paragraph version

The original paper asks whether using machine learning to adjust for confounding breaks the final causal estimate, and whether two fixes — doubly-robust estimators and cross-fitting — repair it. Reproducing it exactly would have taken months of computing, so we agreed a lighter design that asks the same question with far less work. We ran it on 1,000 simulated datasets, and the answer came out clearly: ordinary g-computation is badly biased when paired with machine learning, and the doubly-robust and cross-fit estimators fix it. This document explains the question, the design we agreed, exactly what was run, and the numbers.

Contents

1	The question being asked	2
2	Why we did not just reproduce the paper as-is	2
3	The design we agreed	2
3.1	The three deliberate changes, in plain terms	3
4	What we actually ran	4
4.1	The code	4
4.2	Reproducibility	4
4.3	How it was run	4
5	The results	4
6	What the results mean	5
6.1	Three things worth drawing out	5
6.2	What is stored	5

1 The question being asked

Every method here is trying to estimate one number: the **average treatment effect** — how much a treatment changes the risk of an outcome, if we could give it to everyone versus no one. In this simulated world the true answer is known exactly:

$$\psi_{\text{true}} = -0.108,$$

i.e. the treatment lowers risk by about 10.8 percentage points. Because the data are simulated, we can check each method against this truth.

The treatment is not randomised, so every method must adjust for confounders. That adjustment needs two “nuisance” models: one for who gets treated (the propensity score) and one for the outcome. The interesting choice is *how* to estimate those two models. Here they are estimated by a **super learner** — a machine-learning method that blends several algorithms (a random forest, a neural net, two smoothers, a logistic regression, and a simple mean).

Why this matters (the point of the whole exercise)

There is a well-known worry that machine learning, being flexible, approaches the truth *slowly*, and that this slowness leaks into the causal estimate as bias and as confidence intervals that are too narrow. Theory says two things repair it: (1) use a **doubly-robust** estimator (AIPW or TMLE) instead of a plain plug-in, and (2) use **cross-fitting**, so no data point’s estimate uses a model trained on that same point. The study’s job is to *demonstrate* this, not just assert it.

2 Why we did not just reproduce the paper as-is

The published study runs six estimators, in three modelling set-ups, over 2,000 datasets — and its two heaviest estimators re-fit the super learner hundreds of times per dataset. On a laptop that comes to roughly **1,456 super-learner fits per dataset**, which works out to several months of continuous computing. The authors’ own README says these scripts must be “broken into pieces to run in parallel”; they used a computing cluster.

Rather than chase cluster access, the supervisor proposed a **lighter design that asks the same scientific question** with a small fraction of the computing. That is what we ran.

3 The design we agreed

Six methods, each applied to every dataset. What changed from the paper is shown alongside.

Method	What it is / what changed	SL fits
IPW	Inverse-probability weighting. Unchanged.	1
G-computation (no bootstrap)	Plug-in outcome-model estimator. <i>Change:</i> we drop the 250-fold bootstrap the paper used for its confidence interval, and instead take the interval from the spread of the 1,000 estimates (see below).	1
AIPW	Doubly-robust, no cross-fitting. Unchanged.	2
TMLE	Doubly-robust, no cross-fitting. Unchanged.	2
AIPW, 5-fold cross-fit	<i>Change:</i> replaces the paper’s “double” cross-fit AIPW with ordinary 5-fold cross-fitting, a single split.	10
TMLE, 5-fold cross-fit	<i>Change:</i> replaces the paper’s “double” cross-fit TMLE with ordinary 5-fold cross-fitting, a single split.	10
Total super-learner fits per dataset		26

Compared with the paper’s $\approx 1,456$ fits per dataset, 26 is about a fiftieth — the reason the whole study now fits on a laptop.

3.1 The three deliberate changes, in plain terms

1. G-computation without the bootstrap. Plain g-computation gives only a point estimate, no measure of uncertainty. The paper got an uncertainty estimate by *bootstrapping* (resampling the data 250 times and re-running) — which is what made it expensive. We skip that. Instead we use the fact that this is a simulation: we have 1,000 estimates, so we can *see* how much the estimate actually varies. The standard deviation of those 1,000 estimates is the real standard error — the supervisor calls this the “true SE”. We build g-computation’s confidence interval from that.

2. Simple cross-fitting instead of double. The paper used “double” cross-fitting, an unusual and expensive variant that splits the data three ways and, crucially, trains the two nuisance models on *separate* pieces, repeated 100 times. We use the standard textbook method: split into five folds once, train on four and predict the fifth, rotate through all five. This is what almost everyone means by cross-fitting.

3. A single split. Cross-fitting can be repeated over many random splits and averaged, which slightly steadies the result. We asked whether to do that; the decision was to use a **single split**, because the interest is in summary behaviour over 1,000 datasets, not in squeezing the variance of any one dataset.

What we deliberately kept exactly as published

The **super learner is untouched** — the neural network stays in, the 500-tree random forest is not simplified, and it uses 10-fold internal cross-validation, exactly as the authors set it. The number of datasets was reduced from 2,000 to **1,000**, which the supervisor confirmed is more than enough (the extra precision from 2,000 is negligible).

4 What we actually ran

4.1 The code

The authors' own code was run **completely unmodified** — verified against the published repository. Their estimator classes (IPW, g-computation, AIPW, TMLE) were called from a new driver script we wrote, `reduced_design.py`, which does not touch their files. The two cross-fit methods use `SingleCrossfitAIPW` and `SingleCrossfitTMLE` from **zepid** — the public package written by the paper's own first author — so even the cross-fitting is the author's own implementation rather than something home-made.

Three things the old code needed to run in 2026

The code is from 2019–2020 and breaks in three places on modern libraries (a removed scikit-learn option, a changed statsmodels call, a changed pandas behaviour). Rather than edit the authors' files, we ran everything in a Python environment pinned to era-appropriate library versions, so their source stays byte-for-byte identical. These issues are documented separately.

4.2 Reproducibility

The paper's results cannot be reproduced to the last digit — the random forest and neural net make random choices as they run and the authors set no seed, so even they could not exactly regenerate their own tables. To make *our* results repeatable, we set a fixed seed per dataset. The simulated datasets themselves are fixed (their data-generation step *is* seeded), so we analyse exactly the same 1,000 datasets every time.

4.3 How it was run

Each dataset is independent, so the 1,000 were split into two halves and run side by side on the laptop, using the four processor cores. The run was made robust in three ways, after each failure mode showed up in practice:

- **Resume** — results are written after every dataset, and re-starting skips those already done, so an interruption never costs more than one dataset.
- **Detached** — the jobs run independently of the terminal, so closing it does not stop them.
- **No sleep** — system sleep is held off while on mains power (an early stop was traced to the Mac going to sleep).

The full run took a little over four days of computing and finished all 1,000 datasets with **no failures** (no missing values in any of the six methods).

5 The results

Truth = -0.108 . Each method was run on all 1,000 datasets. “Bias” is how far the average estimate sits from the truth; “coverage” is how often the 95% confidence interval actually contains the truth (it should be about 0.95).

Method	Bias	ESE	ASE	Coverage (own SE)	Coverage (true SE)
IPW	+0.011	0.021	0.023	0.94	0.92
G-computation	+0.026	0.017	—	—	0.64
AIPW	+0.004	0.019	0.017	0.91	0.95
TMLE	−0.001	0.020	0.016	0.90	0.95
AIPW, 5-fold cross-fit	−0.003	0.023	0.022	0.94	0.95
TMLE, 5-fold cross-fit	+0.001	0.020	0.020	0.94	0.96

Bias	average estimate minus truth; should be near 0.
ESE (empirical SE)	how much the estimate really varies across the 1,000 datasets — the “true SE”.
ASE (average SE)	how much variation the method <i>claims</i> , averaged over datasets.
Coverage (own SE)	how often the interval built from the method’s <i>own</i> SE contains the truth.
Coverage (true SE)	how often the interval built from the <i>empirical</i> SE contains the truth.

6 What the results mean

The headline: the paper’s message reproduces

When machine learning does the confounding adjustment, plain **g-computation is the most biased** of the six and its confidence intervals contain the truth only **64%** of the time. The **doubly-robust** methods (AIPW, TMLE) are essentially unbiased, and the **cross-fit** versions are the best behaved — smallest bias and coverage right at 95%. That is exactly the contrast the paper set out to demonstrate.

6.1 Three things worth drawing out

G-computation’s failure is pure bias, not bad variance. Its 64% coverage is measured using the *true* SE — the correct interval width. So the intervals are the right size; they simply sit in the wrong place, because the point estimate is biased. This is the cleanest possible picture of the “plug-in bias” problem: flexible machine learning drags the plug-in estimate away from the truth, and no amount of honest variance estimation rescues it.

Doubly-robust plus cross-fitting is the fix. AIPW and TMLE are near-unbiased even without cross-fitting, and their cross-fit versions land almost exactly on the truth with 95–96% coverage. In this setting they are behaving as the theory promises.

Cross-fitting also improves the *variance* estimate, not just the bias. Look at the “own SE” column for AIPW and TMLE *without* cross-fitting: coverage is about 90%, a little low, because those methods estimate their nuisance models on the same data they then evaluate — so their own standard errors come out slightly too small. The cross-fit versions push that back up to 94%. This point falls out of computing *both* kinds of coverage, and it is a small extra finding beyond the paper’s main story.

6.2 What is stored

Every one of the $1,000 \times 6$ point estimates is saved (with each method’s standard error where it has one), alongside the summary table above. Nothing is thrown away — the full distribution behind each of these six rows is available for any further analysis.

All numbers here are from the completed 1,000-dataset run. The authors' code was run unmodified; the design changes above were agreed with the supervisor and are the only departures from the published study.